Лабораторная работа № 2
Программа для формирования изображений

Цель работы – совершенствование базовых навыков обработки растровых изображений с помощью алгоритмических языков программирования.

Краткие теоретические сведения

В любом визуальном компоненте Lazarus, будь то форма или управляющий элемент, существует специальный объект, средствами которого выполняется рисование видимых частей компонента. Он называется *холстом* и оформлен в виде свойства **Canvas**. В частности, этот объект присутствует в объектах **TImage** и **TBitmap.** Объект **Canvas** имеет пять главных свойств.

1. **Pen: TPen** – объект для рисования линий и границ геометрических фигур:

1) **Color: TColor** – цвет вычерчиваемых пером линий;

2) **Style: TpenStyle** – стиль линий (рис. 1). Учитывается только для толщины линий 1 пиксель. Для толстых линий стиль всегда *psSolid* (сплошная);

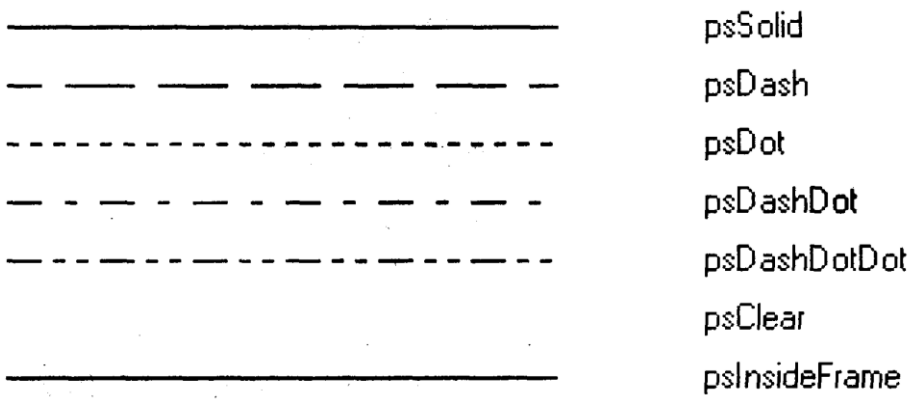


Рис. 1. Стили линий TPen

3) **Width: Integer** – толщина линий в пикселях экрана;

4) **Mode: TPenMode** – способ взаимодействия линий с фоном:

– PmBlack – линии всегда черные. Свойства *Color* и *Style* игнорируются;

– PmWhite – линии всегда белые. Свойства *Color* и *Style* игнорируются;

– PmNop – цвет фона не меняется (линии не видны);

– PmNot – инверсия цвета фона. Свойства *Color* и *Style* игнорируются;

– PmCopy – цвет линий определяется свойством *Color* пера;

– PmNotCopy – инверсия цвета пера. Свойство *Style* игнорируется;

– PmMergePenNot – комбинация цвета пера и инверсионного цвета фона;

– PmMaskPenNot – комбинация общих цветов для пера и инверсионного цвета фона. Свойство *Style* игнорируется;

– PmMergeNotPen – комбинация инверсионного цвета пера и фона;

– PmMaskNotPen – комбинация общих цветов для инверсионного цвета пера и фона. Свойство *Style* игнорируется;

– PmMerge – комбинация цветов пера и фона;

– PmNotMerge – инверсия цветов пера и фона. Свойство *Style* игнорируется;

– PmMask – общие цвета пера и фона;

– PmNotMask – инверсия общих цветов пера и фона;

– PmXor – объединение цветов пера и фона операцией *XOR*;

– PmNotXor – инверсия объединения цветов пера и фона операцией *XOR*.

2. **Brush: TBrush** – объект для заполнения фигур:

1) **Bitmap: TBitmap** – растровое изображение, которое будет использоваться кистью для заполнения. Если это свойство определено, свойства *Color* и *Style* игнорируются;

2) **Color: TColor** – цвет кисти;

3) **Style: TBrushStyle** – стиль кисти (рис. 2).

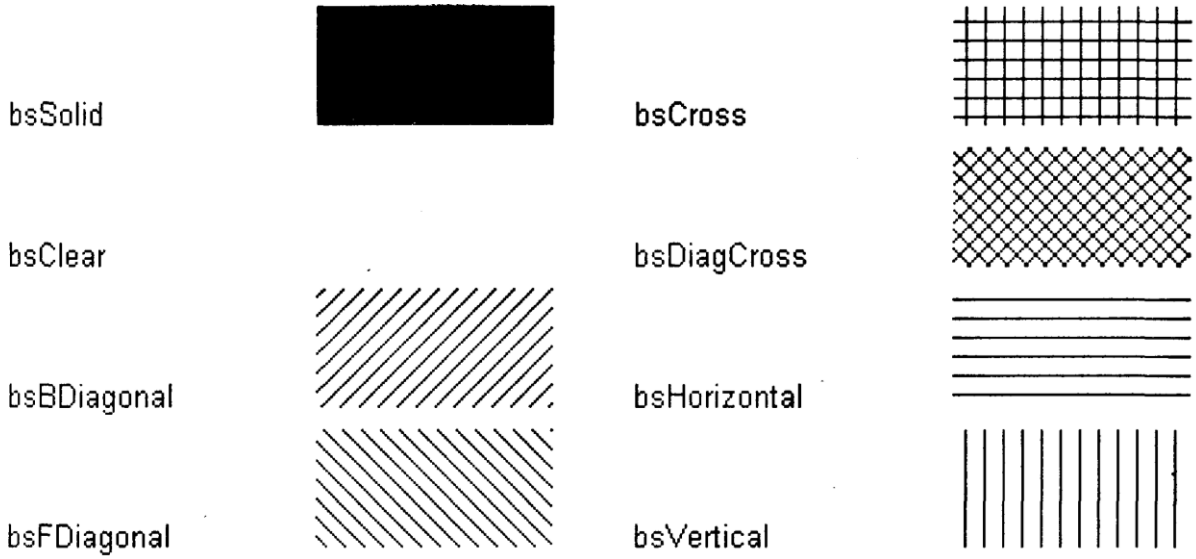


Рис. 2. Стили заполнения TBrush

3. **Font: TFont** – объект для вывода текста:

1) **Color: TColor** – цвет шрифта;

2) **Height: Integer** – высота шрифта в пикселях экрана;

3) **Name: TFontName** – имя шрифта. По умолчанию имеет значение *MS Sans Serif*;

4) **Pitch: TFontPitch** – способ расположения букв в тексте: значение *fpFixed* задаёт моноширинный текст, при котором каждая буква имеет одинаковую ширину; значение *fpVariabel* определяет пропорциональный текст, при котором ширина буквы зависит от её начертания; *fpDefault* определяет ширину, принятую для текущего шрифта;

5) **PixelPerlnch: Integer** – количество пикселей экрана на один дюйм реальной длины. Это свойство не следует изменять, т. к. оно используется системой для обеспечения соответствия экранного шрифта шрифту принтера;

6) **Size: Integer** – высота шрифта в пунктах (1/72 дюйма). Изменение этого свойства автоматически изменяет свойство *Height*, и наоборот;

7) **Style: TFontStyles** – стиль шрифта. Может принимать значение как комбинацию следующих признаков: *fsBold* (жирный), *fsItalic* (курсив), *fsUnderline* (подчёркнутый), *fsStrikeOut* (перечёркнутый).

4. **PenPos: TPoint** – объект для хранения текущей позиции рисования.

5. **Pixels[X, Y: Integer]: TColor** – объект для хранения пикселов холста.

Свойства **TPen, TBrush, TFont,** представляют собой инструменты для рисования. Сами по себе эти объекты не рисуют, но хранят в своих свойствах необходимые холсту параметры рисования.

Холст поддерживает такое понятие, как текущая позиция рисования. Текущая позиция хранится в свойстве **PenPos** и используется при рисовании прямых (прямая рисуется от текущей позиции до заданной). Выражение **PenPos.X** возвращает горизонтальную позицию, а **PenPos.Y** – вертикальную.

Благодаря свойству **Pixels** холст интерпретируется как двумерная матрица пикселов. Элемент, стоящий на пересечении столбца **X** и строки **Y** матрицы **Pixels**, кодирует цвет пиксела, который задаётся объектом типа **TColor**. Левый верхний пиксел изображения имеет координаты [0,0], ось **X** направлена вправо, а ось **Y** – вниз.

Все методы можно разбить на несколько групп в соответствии с их назначением.

1. Рисование прямых, ломаных и кривых линий;

1) **MoveTo(X, Y: Integer)** – перемещает указатель текущей позиции в заданную точку.

2) LineTo(X, Y: Integer) – рисует прямую линию от текущей позиции рисования **(PenPos.X, PenPos.Y)** до заданной **(X, Y)** и перемещает указатель текущей позиции в точку с координатами **(X, Y).**

3) **Polyline(Points: array of TPoint)** – рисует ломаную линию, соединяя точки массива **Points**. Для получения элемента массива по двум координатам может использоваться функция **Point(X, Y: Integer): TPoint.** Пример: с помощью метода **Polyline** можно нарисовать контур треугольника:

```
with Canvas do
  begin
    Pen.Style:=psDot; //Использовать пунктирную линию
    Brush.Color:= RGB(0,255,255); // васильковый цвет
    //Нарисовать треугольник)
    Polyline([Point(20,90),Point(90,20),Point(180,90),Point(20,90)]);
  end;
```

4) Arc(X1, Y1, X2, Y2, X3, Y3, X4, Y4: Integer) – рисует дугу эллипса, вписанного в прямоугольник с координатами **(X1, Y1)** и **(X2,**

Y2). Дуга определяется двумя радиусами эллипса, приходящимися через точки **(X3, Y3)** и **(X4, Y4).** Дуга рисуется против часовой стрелки от точки пересечения эллипса с первым радиусом до точки пересечения со вторым радиусом (рис. 3);

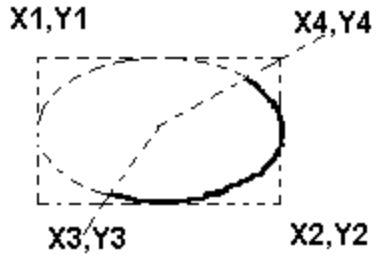


Рис. 3. Схема рисования дуги эллипса

5) **Chord(X1, Y1, X2, Y2, X3, Y3, X4, Y4: Integer)** – рисует хорду эллипса, вписанного в прямоугольник с координатами **(X1, Y1)** и **(X2, Y2)** Хорда лежит на прямой, проходящей через точки **(X3, Y3)** и **(X4, Y4)** (рис. 4).

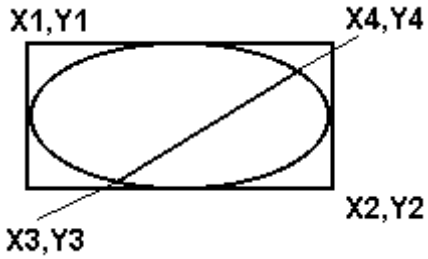


Рис. 4. Схема рисования хорды эллипса

2. Рисование геометрических фигур:

1) **Rectangle(X1, Y1, X2, Y2: Integer)** – рисует прямоугольник с левым верхним углом в точке **(X1, Y1)** и нижним правым углом в точке **(X2, Y2)**. Прямоугольник рисуется текущими атрибутами кисти и пера;

2) RoundRect(X1, Y1, X2, Y2, X3, Y3: Integer) – рисует прямоугольник с закруглёнными углами. Углы рисуются как четверти эллипса с шириной **X3** и высотой **Y3**. (рис. 5);

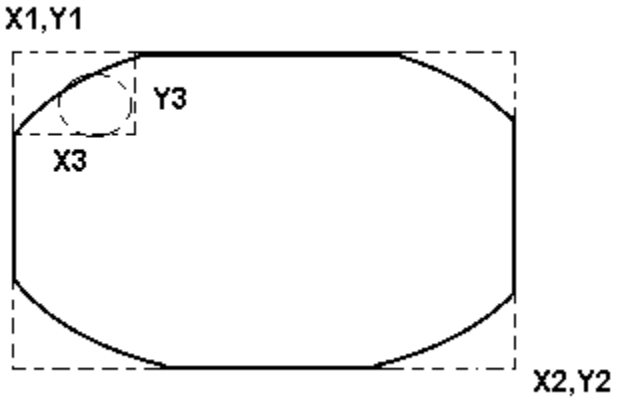


Рис. 5. Схема рисования прямоугольника с закруглёнными углами

3) Ellipse(X1, Y1, X2, Y2: Integer) – рисует эллипс, вписанный в прямоугольник с левым верхним углом в точке **(X1, Y1)** и нижним правым углом в точке **(X2, Y2)** (рис. 6). Рисуется текущими пером и кистью;

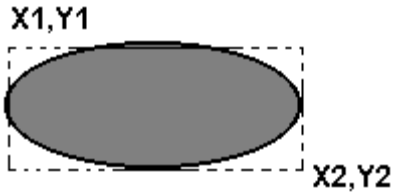


Рис. 6. Схема рисования эллипса

4) **Polygon(Points: array of TPoint)** – рисует ломаную линию, соединяя точки массива **Points**. Ломаная замыкается отрезком из последней точки в первую, и полученная фигура заполняется текущей кистью;

5) Pie(X1, Y1, X2, Y2, X3, Y3, X4, Y4: Integer) – рисует сектор эллипса, вписанного в прямоугольник с координатами **(X1, Y1)** и **(X2, Y2)** (рис. 7). Сектор определяется двумя радиусами эллипса, проходящими через точки **(X3, Y3)** и **(X4, Y4).**

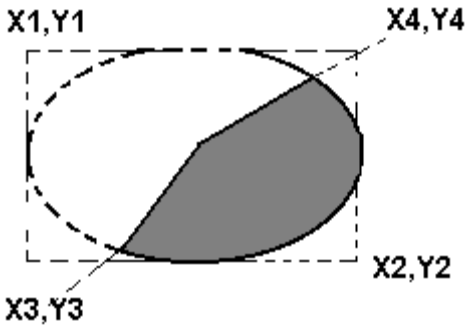


Рис. 7. Схема рисования сектора эллипса

Пример: эллипс, вписанный в холст.

```
with Canvas do
  begin
    Pen.Width:=10; //Использовать очень толстое перо
    Pen.Style:=psInsideFrame; //и вписать фигуру
    Ellipse(0,0,Width,Height); //Нарисовать вписанный эллипс
  end;
```

3. Заполнение внутреннего пространства фигур:

1) FillRect(const Rect: TRect) – заполняет прямоугольник, используя текущую кисть. Для получения прямоугольника по его координатам может использоваться функция **Rect(X1, Y1, X2, Y2: Integer): TRect)**;

2) FloodFill(X, Y: Integer; Color: TColor; FillStyle: TFillStyle) – заливает область экрана, используя атрибуты кисти. Заливка начинается в точке **(X, Y)** и продолжается во всех направлениях до достижения цветовой границы. Способ заполнения области определяется параметром **FillStyle**. Если параметр **FillStyle** равен *fsBorder*, заливка заполняется до тех пор, пока не обнаружится граница с цветом, указанным в параметре **Color**. Если же параметр **FillStyle** равен *fsSurface*, заливка выполняется, пока не обнаруживается цвет, указанный в параметре **Color.**

4. Вывод текста:

1) TextOut(X, Y: Integer; const Text: string) – выводит текущим шрифтом строку текста Text в прямоугольнике с левым верхним углом в точке **(X, Y)**;

2) TextRect(Rect: TRect; X, Y: Integer; const Text: string) – выводит текст в прямоугольнике **Rect.** Вывод за границы прямоугольника отсекается (см. метод **TextOut**);

3) **TextHeight(const Text: string): Integer** – возвращает высоту (в пикселах) строки **Text** при выводе её текущим шрифтом;

4) TextWidth(const Text: string): Integer – возвращает ширину (в пикселах) строки **Text** при выводе её текущим шрифтом.

5. Вывод картинок:

1) **Draw(X, Y: Integer; Graphic: TGraphic)** – выводит картинку **Graphic** в точке **(X, Y)**. **TGraphic** – это абстрактный базовый класс для классов **TBitMap**, **TIcon**, **TMetaFile**, реализующих картинки конкретных форматов. Класс **TGraphic** как контейнер **Graphic** включен, в частности, в класс **TPicture**, так что, если есть объект **P: TPicture**, можно получить **P.Graphic: TGraphic**;

2) **StretchDraw(const Rect; TRect; Graphic: TGraphic)** – выводит картинку **Graphic**, масштабируя её до прямоугольника **Rect**;

3) **CopyRect(Dest: TRect; Canvas: TCanvas; Source: TRect)** – копирует прямоугольную область **Source** с холста **Canvas**. Результирующая картинка масштабируется до прямоугольника **Dest**;

4) Способ комбинации точек исходной картинки с точками холста определяется свойством холста **CopyMode**. Возможные значения этого свойства (**FinalPixel** означает цвет результирующего пиксела; **SrcPixel** – цвет пиксела копируемой картинки; **DestPixel** – цвет исходного пиксела полотна; **BrushPixel** – цвет кисти полотна):

- CmBlackness – всё результирующее изображение чёрное;
- CmWhiteness – всё результирующее изображение белое;
- CmSrcCopy – FinalPixel := SrcPixel;
- CmSrcPaint – FinalPixel := SrcPixel or DestPixel;
- CmSrcAnd – FinalPixel := SrcPixel and DestPixel;
- CmSrclnvert – FinalPixel := SrcPixel xor DestPixel;
- CmSrcErase – FinalPixel := SrcPixel and (not DestPixel);
- CmNotSrcCopy – FinalPixel := not SrcPixel;
- CmNotSrcErase – FinalPixel := not (SrcPixel or DestPixel);
- CmDstlnvert – FinalPixel := not DestPixel;
- CmMergePaint – FinalPixel := (not SrcPixel) or DestPixel;
- CmMergeCopy – FinalPixel := BrushPixel) and SrcPixel;
- CmPatCopy – FinalPixel := BrushPixel;

– CmPatPaint – FinalPixel := BrushPixel or (not SrcPixel) or DestPixel;

– CmPatlnvert – FinalPixel := BrushPixel xor DestPixel;

Задание

1. Разработать алгоритм и программу для чтения из файла, преобразования и записи в файл графической информации по варианту. Программа включает в себя:

- создание нового изображения;
- чтение изображения из файла;
- добавление фрагмента заданной формы из заданной позиции изображения, загруженного из файла в заданную позицию созданного нового изображения попиксельно;
- добавление осей системы координат и графика функции;
- сохранение созданного изображения в файл на диске.

2. Ответить на контрольные вопросы.
3. Оформить отчёт.

Содержание отчёта

1. Титульный лист.
2. Формулировка задачи.
3. Схема алгоритма.
4. Программа на алгоритмическом языке программирования.
5. Форма.
6. Исходное и преобразованное изображения.
7. Ответы на контрольные вопросы.

Порядок выполнения работы

1. Ознакомление с целью работы, заданием, содержанием отчёта, порядком выполнения работы, индивидуальным вариантом, примером выполнения работы.

2. Ознакомление со средствами системы программирования для чтения, обработки и записи графики на примере и с помощью технической документации.

3. Разработка алгоритма в соответствии с вариантом задания.

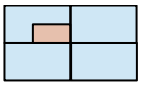
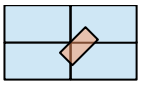
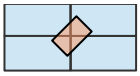
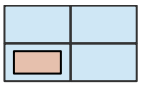
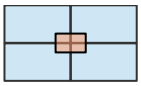
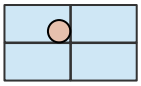
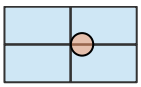
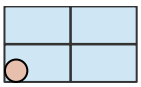
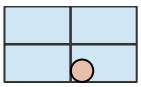
4. Разработка и отладка программы для решения задачи.
5. Обработка изображения.
6. Ответы на контрольные вопросы.
7. Оформление отчёта.

Индивидуальные задания

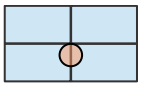
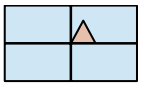
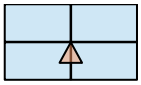
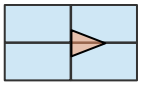
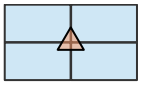
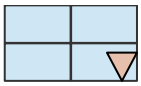
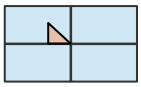
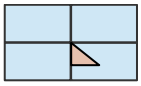
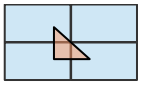
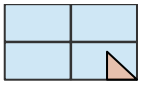
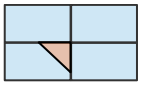
Каждый вариант индивидуальных заданий (табл. 1) включает в себя графическое изображение формы и позиции фрагмента на исходном изображении, размеченном проходящими через центр вертикальной и горизонтальной линиями, позицию для размещения фрагмента на новом изображении и функцию, график которой нужно нарисовать на новом изображении. Конкретные числовые параметры, детально определяющие форму и положение фрагментов, выбираются произвольно, но так чтобы не нарушить содержание условия задания.

Таблица 1

Варианты индивидуальных заданий

№	Форма и позиция фрагмента на исходном изображении	Регион размещения на новом изображении	Функция
1		Левый верхний угол	sin(x)
2		Левый нижний угол	cos(x)
3		Правый верхний угол	tan(x)
4		Правый нижний угол	x*sin(x)
5		Центр	exp(x)
6		Левый край по центру	ln(x)
7		Правый край по центру	x^2
8		Верхний край по центру	x^3
9		Нижний край по центру	x^(1/2)

Окончание табл. 1

№	Форма и позиция фрагмента на исходном изображении	Регион размещения на новом изображении	Функция
10		В заданную позицию	x^(1/3)
11		Левый верхний угол	1/x
12		Левый нижний угол	(1/x)^2
13		Правый верхний угол	1/(x-1)^2
14		Правый нижний угол	sin(x^2)
15		Центр	1/exp(x)
16		Левый край по центру	|x-1|
17		Правый край по центру	x*cos(x)
18		Верхний край по центру	x^(3/7)
19		Нижний край по центру	sin^2(x)
20		В заданную позицию	2^x

Контрольные вопросы

1. В чём основные отличия свойств оптического излучения от других видов излучения?

2. Почему зрение так важно для человека? Чем его можно заменить для восприятия той же информации об окружающем мире?

3. Почему человек воспринимает излучение в пределах именно этой небольшой части оптического диапазона, называемой видимым светом?

4. Опишите характеристики трёх самосветящихся источников, которые чаще всего попадаются на глаза.

5. В каких случаях имеет смысл корректировать спектр источника излучения?

6. Как получить анизотропный диффузный рефлектор и как сделать его изотропным?

7. Почему функция яркости непрерывна, в то время как количество излучаемых фотонов дискретно?

8. Опишите достоинства и недостатки каждого из способов разделения света на зоны.

9. Какой из способов размещения чувствительных элементов фотоприёмника предпочтителен – регулярный (как на матрице) или хаотический (как на сетчатке глаза)?

10. Почему оптическая система глаза, состоя из небольшого количества элементов, обеспечивает сравнительно высокое качество изображения?

11. Какими параметрами должен обладать искусственный датчик излучения, чтобы не уступать глазу?

12. Какие параметры, влияющие на дискретизацию и квантование, следует улучшать в первую очередь?

13. Какова система координат холста?

14. Как с помощью программы вывести на холст движущиеся изображения?

Пример выполнения работы

Задание:

1. Создание нового изображения.
2. Чтение изображения из файла.
3. Добавление фрагмента формы круг из заданной позиции изображения, загруженного из файла в заданную позицию созданного нового изображения попиксельно.
4. Добавление осей системы координат и графика функции sinc(x).
5. Сохранение созданного изображения в файл на диске.

В соответствии с логикой выполнения действий, указанных в задании, пользователь должен выполнить следующие действия:

1. Задать ширину и высоту создаваемого изображения в пикселях.

2. Создать новое изображение заданного размера.
3. Открыть файл с имеющимся изображением.
4. Задать координаты центра и радиус круга, который нужно скопировать на новое изображение.
5. Задать координаты размещения фрагмента на новом изображении.
6. Скопировать фрагмент.
7. Нарисовать оси системы координат.
8. Нарисовать график функции, определив масштаб.
9. Сохранить результат в файл на диске.

В соответствии с составом и последовательностью действий, ожидаемых от пользователя, разместим на форме (рис. 8) следующие компоненты: **TOpenPictureDialog, TSavePictureDialog, 2 TImage, 6 Button, 7 TEdit, 11 Label**. Для **TImage** сразу зададим нужные значения свойствам **Proportional** и **Stretch**. Добавим текст на кнопки, изменив соответствующие свойства. Разработаем алгоритмы обработчиков событий для нажатия кнопок и закрытия формы. Рассчитывая эту программу для чтения любых типов графических файлов и получения полноцветного результата, добавим функцию ConvertTo24Bit преобразования в формат пикселей RGB.

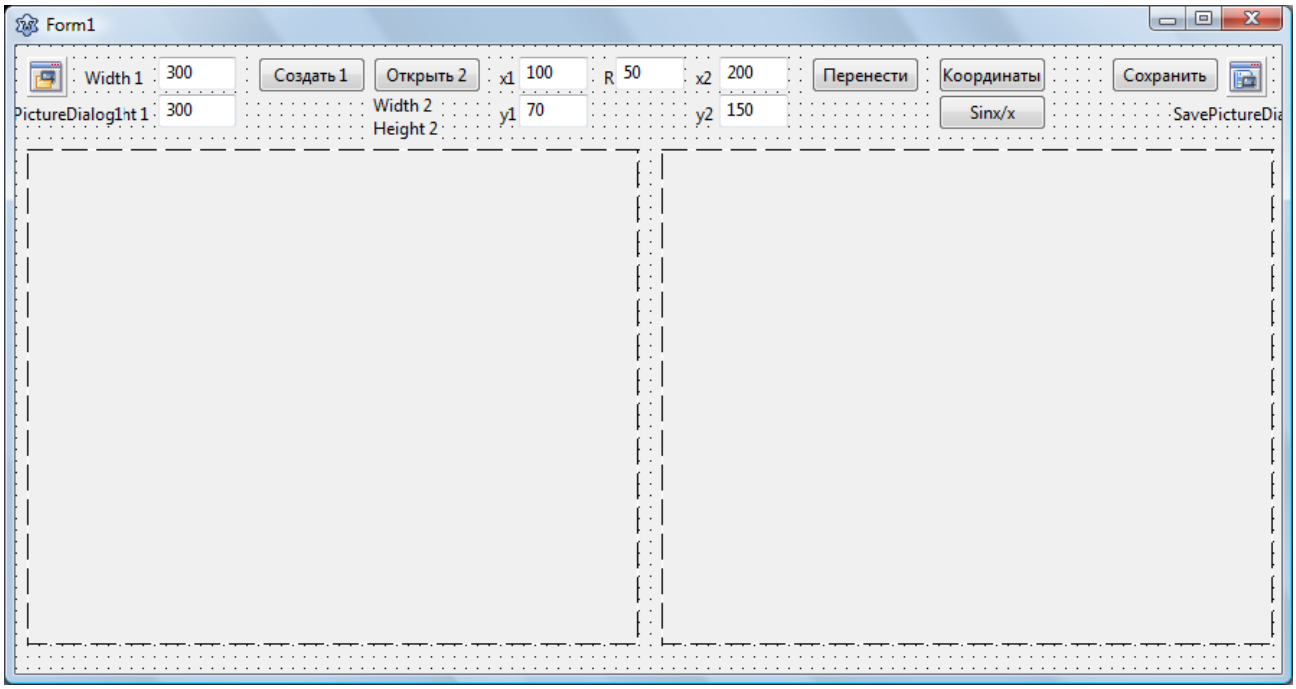


Рис. 8. Форма программы

При нажатии на кнопку «Создать 1» для битмапа Image1 выполняется очистка, задаётся формат пикселей pf24bit, из соответствующих компонентов Edit считывается ширина и высота, выполняется заполнение белым фоном.

При нажатии на кнопку «Открыть 2» выполняется чтение изображения из файла. Любое считанное изображение переводится в полноцветный формат. Если формат считанного изображения неполноцветный, вызывается процедура преобразования и выводится сообщение.

Для копирования круга считываем координаты размещения центра круга на новом изображении x1, y1, координаты центра круга на источнике x2, y2 и радиус R. Вычисляем разницу между координатами центра dx, dy. Рассматривая в цикле все точки нового изображения, проверяем условие попадания внутрь нового круга. Если оно выполняется, с учётом разности координат проверяем условие попадания в исходное изображение. Если не выходим за пределы нового изображения, считываем три составляющих пикселя исходного изображения и записываем их в результат, иначе в результат записываем пиксели чёрного цвета. Если попадаем не внутрь круга, а на границу, выводим точку определённого цвета в новое и исходное изображение.

График будем рисовать в правой полуплоскости. Функция принимает положительные и отрицательные значения, так что ось OX располагаем по центру. Деления намечаем через каждые десять пикселей. Выводим заголовок. Цвет графика задаём светло-синий. Выбираем масштаб по OX: единица – каждое деление, т. е. десять пикселей. Для соответствия этому масштабу при вычислении значения аргумента функции в позиции каждого пикселя номер пикселя нужно разделить на 10. Максимальное значение функции sin(x)/x – 1 при x=0. Выбираем масштаб по OY – единица на верхнем делении. Верхнее деление находится в позиции (10,20). Так как ось OX (и 0 на OY) расположены на уровне середины графика (height div 2), а верхнее деление – на 20 пикселей ниже верхней границы, единица будет занимать height div 2 – 20 пикселей, и для соответствия выбранному масштабу значение функции нужно умножить на это число.

Сохраняется изображение как в предыдущей работе.

Кодируем. Получаем модуль примерно следующего содержания:

```
unit Unit1;

{$mode objfpc}{$H+}

interface

uses
  Classes, SysUtils, FileUtil, Forms, Controls, Graphics, Dialogs, StdCtrls,
  GraphType, IntfGraphics, LCLType, LCLProc, LCLIntf, ExtCtrls,
  ExtDlgs, fpImage, crt, math;

type

  { TForm1 }

  TForm1 = class(TForm)
    Button1: TButton;
    Button2: TButton;
    Button3: TButton;
    Button4: TButton;
    Button5: TButton;
    Button6: TButton;
    Edit1: TEdit;
    Edit2: TEdit;
    Edit3: TEdit;
    Edit4: TEdit;
    Edit5: TEdit;
    Edit6: TEdit;
    Edit7: TEdit;
    Image2: TImage;
    Label1: TLabel;
    Label10: TLabel;
    Label11: TLabel;
    Label12: TLabel;
    Label13: TLabel;
    Label3: TLabel;
```

```
    Label5: TLabel;
    Label6: TLabel;
    Label7: TLabel;
    Label8: TLabel;
    Label9: TLabel;
    OpenPictureDialog1: TOpenPictureDialog;
    SavePictureDialog1: TSavePictureDialog;
    Image1: TImage;
    procedure Button1Click(Sender: TObject);
    procedure Button2Click(Sender: TObject);
    procedure Button3Click(Sender: TObject);
    procedure Button4Click(Sender: TObject);
    procedure Button5Click(Sender: TObject);
    procedure Button6Click(Sender: TObject);
    procedure FormClose(Sender: TObject; var CloseAction:
TCloseAction);
  private
    { private declarations }
  public
    { public declarations }
  end;

var
  Form1: TForm1;

implementation

{$R *.lfm}

{ TForm1 }
//Функция преобразования в pf24bit
function ConvertTo24Bit(aValue: TBitmap;BackColor: TColor): TBitmap;
begin
  Result := Tbitmap.Create;//Преобразовывается bitmap avalue в result
  with Result do
    if aValue.PixelFormat <> pf24bit then //если исходный bitmap не 24bit
      begin
        PixelFormat:= pf24bit; //формат результата 24 бита
```

```
      SetSize(aValue.Width,aValue.Height); //размер result как у avalue
      Canvas.FloodFill(0,0,BackColor,fsBorder); //заливаем result цветом
      Canvas.Draw(0,0,aValue); //выводим в result картинку из avalue
    end
  else //если исходный битмап 24bit
    Assign(AValue); //результат полагаем соответствующим исходнику
end;

procedure TForm1.Button1Click(Sender: Tobject); //Кнопка Создать 1
begin
//С битмапом расположенного на форме TImage
 with Image1.Picture.Bitmap do
  begin
    Clear; //а вдруг там мусор
    PixelFormat:=pf24bit; //формат пикселей
    Width:=StrToInt(Edit6.Text); //Ширина из Edit
    Height:=StrToInt(Edit7.Text); //Высота из Edit
    Canvas.FloodFill(0,0,RGB(255,255,255),fsBorder); //Белый фон
  end;
end;

procedure TForm1.Button4Click(Sender: Tobject); //Кнопк Открыть 2
begin
 if OpenPictureDialog1.Execute then
   Image2.Picture.LoadFromFile(OpenPictureDialog1.FileName);
 Label6.Caption:=IntToStr(Image2.Picture.Width); //Для справки
 Label8.Caption:=IntToStr(Image2.Picture.Height);//выводим размер
//Если формат пикселей не такой как надо
 if Image2.Picture.bitmap.PixelFormat <> pf24bit then
   begin
     showmessage('invalid pixel color format. trying to convert');
image2.picture.bitmap.assign(convertto24bit(image2.Picture.bitmap,RGB(
255,255,255))); //воспользуемся функцией преобразования
   end;
end;

procedure TForm1.Button2Click(Sender: Tobject); //Перенести
 var
```

```
  cr,cb,cg:byte;
  px, py,x1,y1,x2,y2,r,dx,dy: Integer;
  begin
  x1:=strtoint(edit1.text);//Считываем параметры окружности
  x2:=strtoint(edit4.text);//и координат переноса
  y1:=strtoint(edit2.text);
  y2:=strtoint(edit5.text);
  r:=strtoint(edit3.text);
  dx:=x1-x2;
  dy:=y1-y2;
  for py:=0 to Image1.picture.bitmap.Height-1 do begin//для всех строк
     for px:=0 to Image1.picture.bitmap.Width-1 do begin//для пикселей
//Если попадает в окружность
       if (round(sqrt(power(py-y1,2)+power(px-x1,2))) < r) then
//Если не выходит за пределы изображения
        if ((px-dx)>0) and ((py-dy)>0) and ((px-dx)<image2.Picture.Width)
and ((py-dy)<image2.Picture.Height) then
          begin //Считываем три составляющих
        cr:=getrvalue(Image2.Picture.bitmap.canvas.pixels[px-dx,py-dy]);
        cg:=getgvalue(Image2.Picture.bitmap.canvas.pixels[px-dx,py-dy]);
        cb:=getbvalue(Image2.Picture.bitmap.canvas.pixels[px-dx,py-dy]);
//Записываем куда надо
           Image1.Picture.bitmap.Canvas.pixels[px,py]:=RGB(cr,cg,cb);
          end
//Если не откуда считать
        else Image1.Picture.bitmap.Canvas.pixels[px,py]:=RGB(0,0,0)
       else if (round(sqrt(power(py-y1,2)+power(px-x1,2))) = r) then
        begin
//Обводим контуром то, что скопировали для красоты.-)
     Image1.Picture.bitmap.Canvas.pixels[px,py]:=RGB(127,255,127);
          with Image2.Picture.Bitmap do
            if ((px-dx)>0)and((py-dy)>0)and((px-dx)<Width)and((py-
dy)<Height) then //Обводим контуром исходный фрагмент для справки
             Canvas.pixels[px-dx,py-dy]:=RGB(255,127,127);
         end;
     end;
   end;
end;
```

```
procedure TForm1.Button6Click(Sender: Tobject); //Координаты
var i:integer;
begin //Рисуем декартову сетку из отрезков
  With Image1.Picture.bitmap do //на битмапе новой картинки
    begin
      h:=height; //сохраняем высоту
      w:=width;//сохраняем ширину
      with canvas do //на холсте битмапа новой картинки
        begin
          Polyline([point(10,h-10),point(10,10)]); //ось OY
          Polyline([point(5,20),point(10,10),point(15,20)]);//стрелочка OY
          Polyline([point(8,h div 2),point(w - 10, h div 2)]);//ось OX
          Polyline([point(w-20,h div 2-5),point(w-10,h div 2),point(w-20,h
div 2+5)]); //стрелочка OX
          for i:=0 to (h - 40) div 20 do//рисуем деления на OY
            begin//относительно центра оси, размером 2 с шагом 10 каждое
             Polyline([point(10,i*10 + h div 2),point(8, i*10 + h div 2)]);//вниз
             Polyline([point(10,h div 2 - i*10),point(8,h div 2 - i*10)]);//вверх
            end;
          for i:=0 to (w - 40) div 10 do //рисуем деления на OX с шагом 10
            Polyline([point((i+1)*10,h div 2),point((i+1)*10,h div 2 + 2)]);
          textout(1,h div 2 +1,'0');//подписываем 0
          TextOut(w - 20, h div 2 + 6, 'x');//подписываем O
          textout(20,1,'y');//подписываем ось OY
        end;
    end;
end;

procedure TForm1.Button5Click(Sender: TObject);
var i:integer;
begin
  With Image1.Picture.bitmap do //рисуем на новом битмапе
    begin
      canvas.textout(width div 2,1,'y=sin(x)/x'); //заголовок графика
      canvas.Pen.Color:=RGB(127,127,255); //цвет точек
      canvas.moveto(10,20 ); //первая точка (с учётом масштаба) на оси
      for i:=1 to width - 20 do //оси OY до конца оси OX со сдвигом
```

```
//рисуем отрезками с учётом масштаба по вертикали и горизонтали
  canvas.lineto(i+10,height div 2-round(sin(i/10)/(i/10)*(height div 2-20)));
    end;
end;

procedure TForm1.Button3Click(Sender: Tobject);//Сохранить
begin
  if SavePictureDialog1.Execute then
       Image1.Picture.SaveToFile(SavePictureDialog1.FileName);
end;

procedure TForm1.FormClose(Sender: TObject; var CloseAction:
TCloseAction);
begin
       Image1.Free; //освобождаем память
       Image2.Free;
end;

end.
```

Запускаем программу (рис. 9).

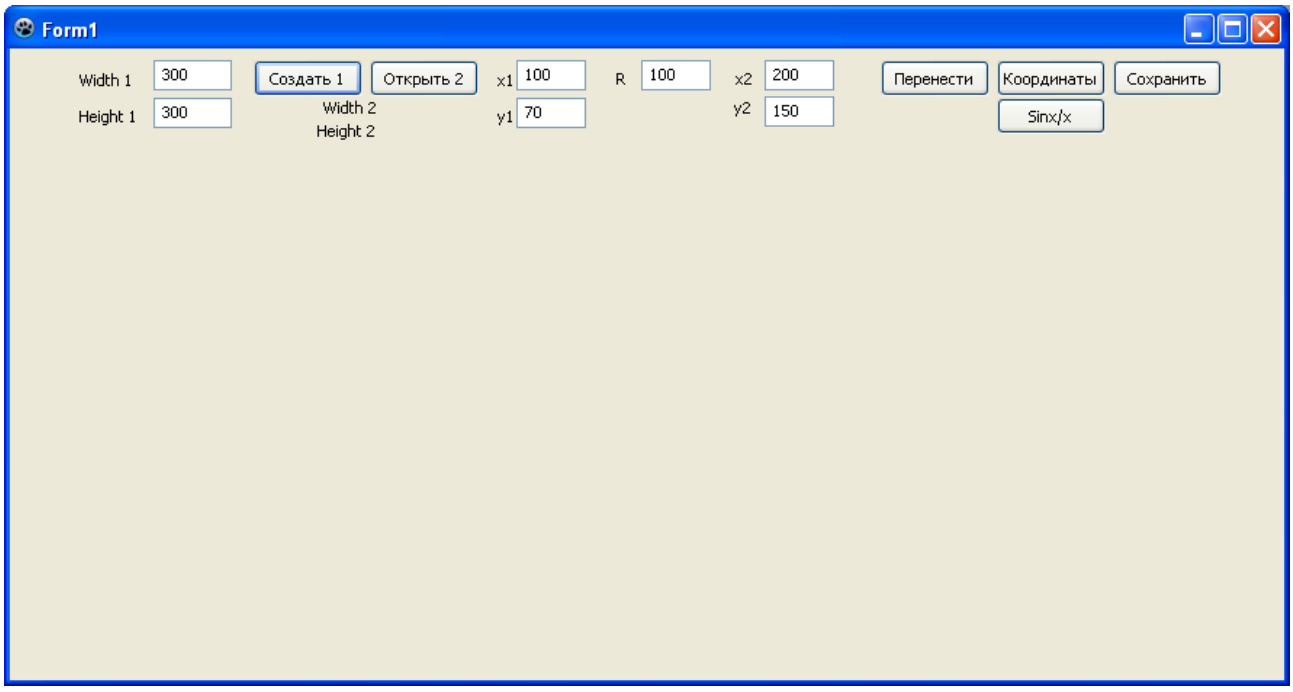


Рис. 9. Окно программы после запуска

Создаём изображение 300x300 пикселей (оно заливается белым) (рис. 10).

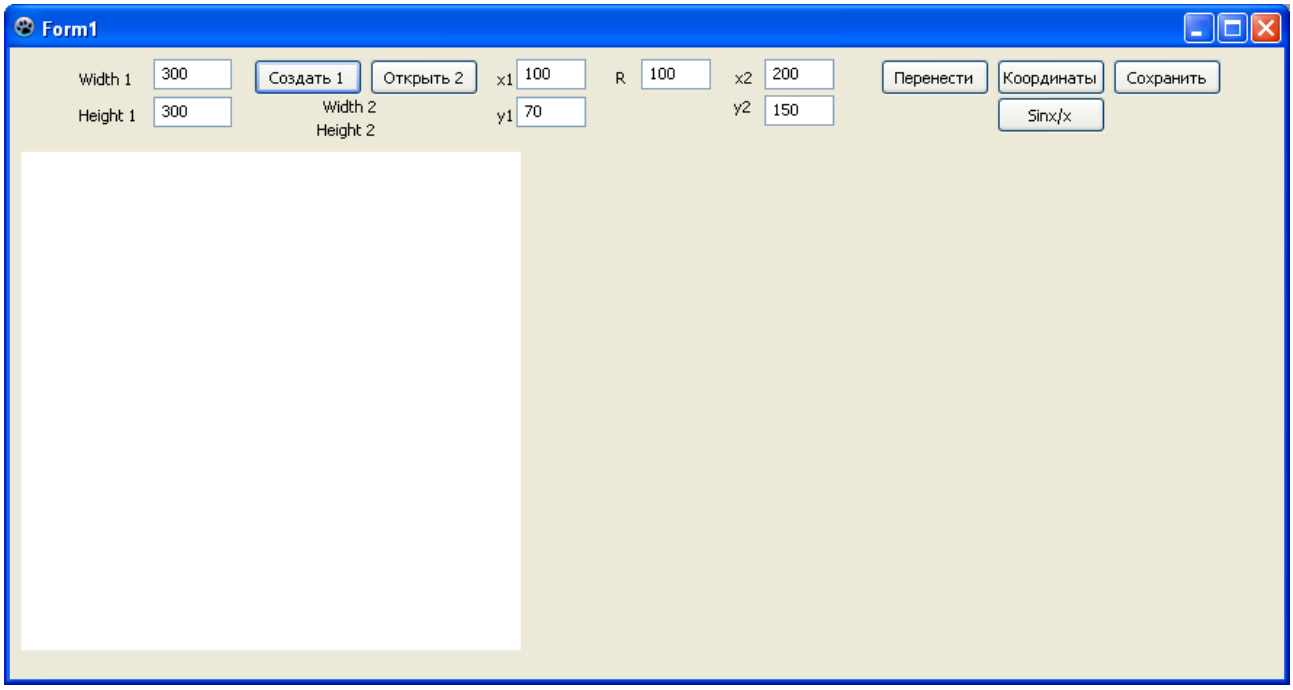


Рис. 10. Окно с созданным новым изображением

Открываем изображение из файла (рис. 11).

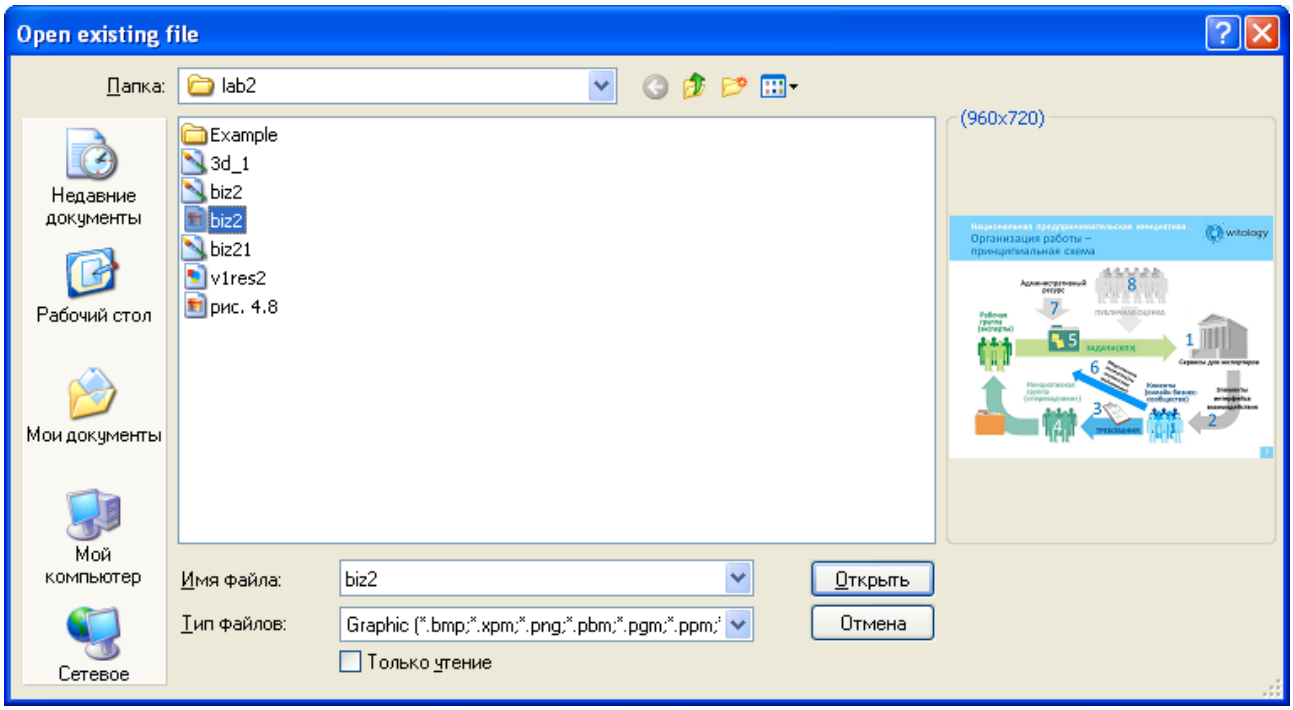


Рис. 11. Диалоговое окно выбора файла изображения

Так как оно не 24–битное, выдаётся сообщение о преобразовании (рис. 12).

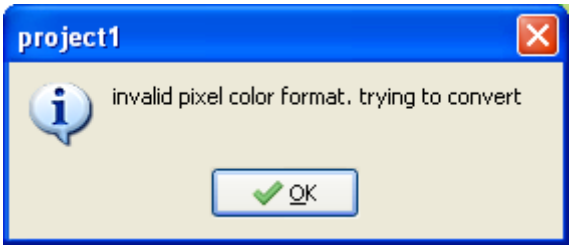


Рис. 12. Сообщение о преобразовании

Соглашаемся. Изображение считывается на форму (рис. 13).

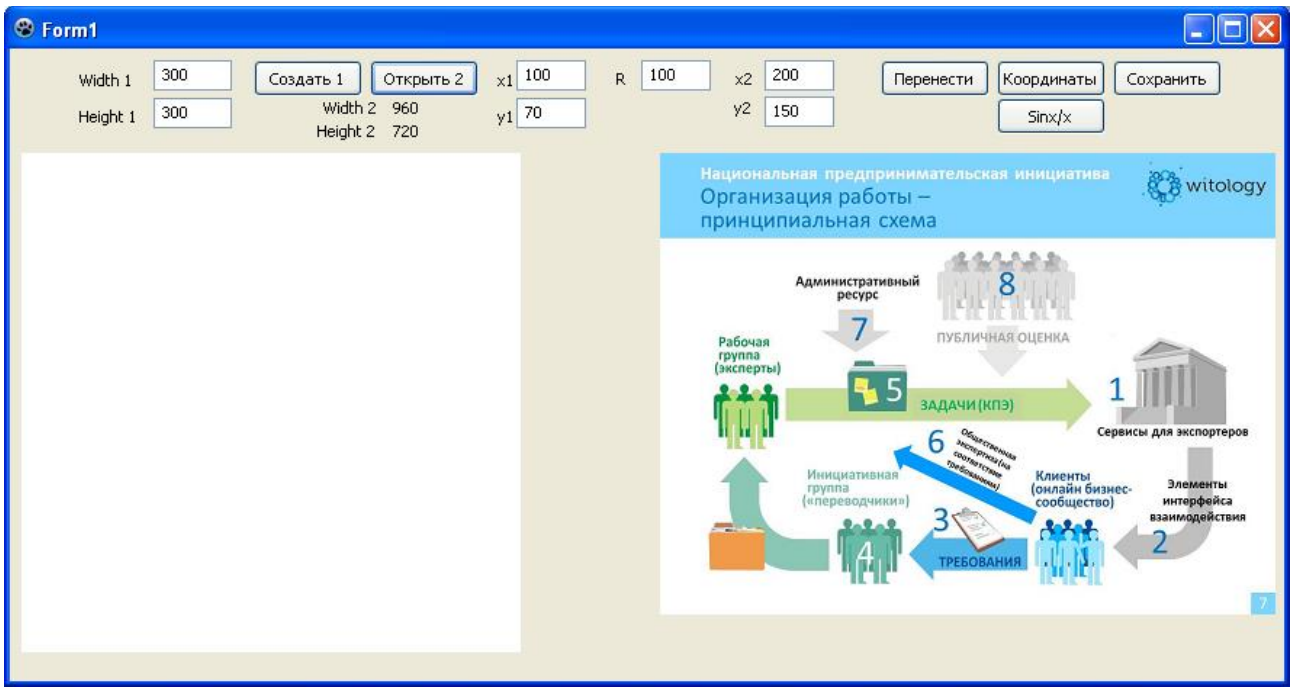


Рис. 13. Окно с загруженным из файла изображением

Переносим круговой фрагмент изображения радиусом 100 пикселей из позиции (200,150) в позицию (100,70) нового изображения (рис. 14).

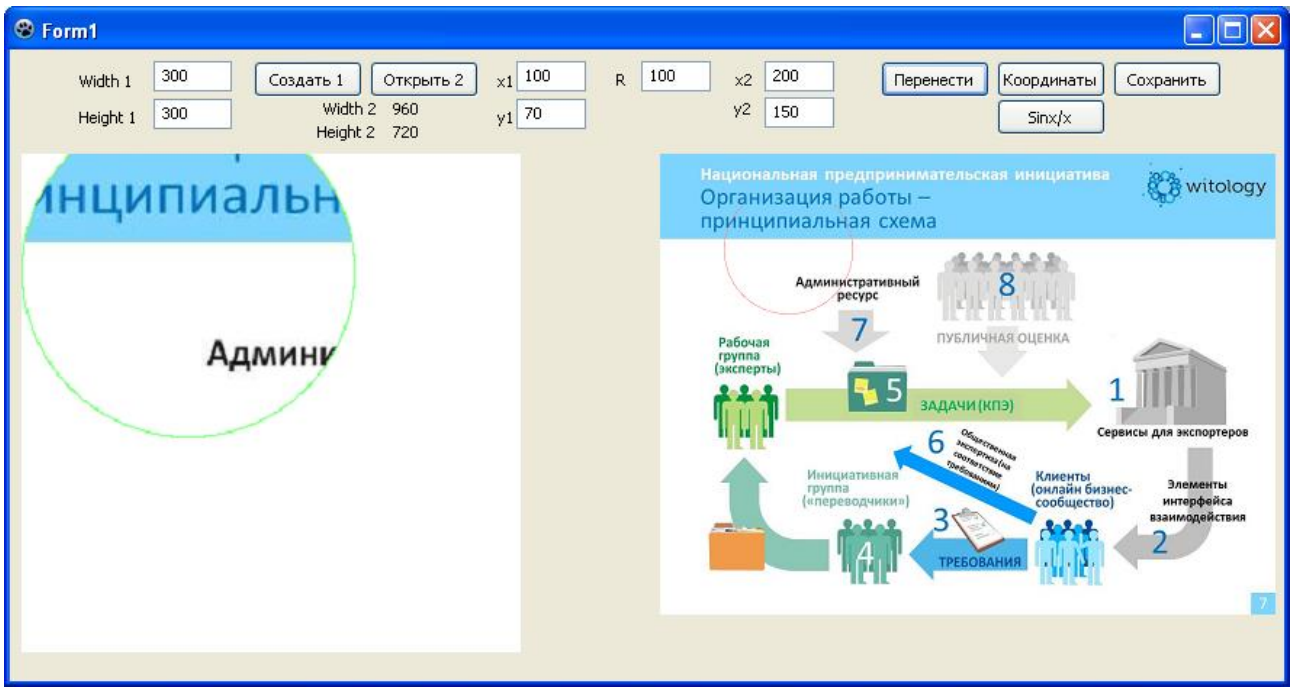


Рис. 14. Перенос фрагмента изображения

Рисуем координатные оси (рис. 15).

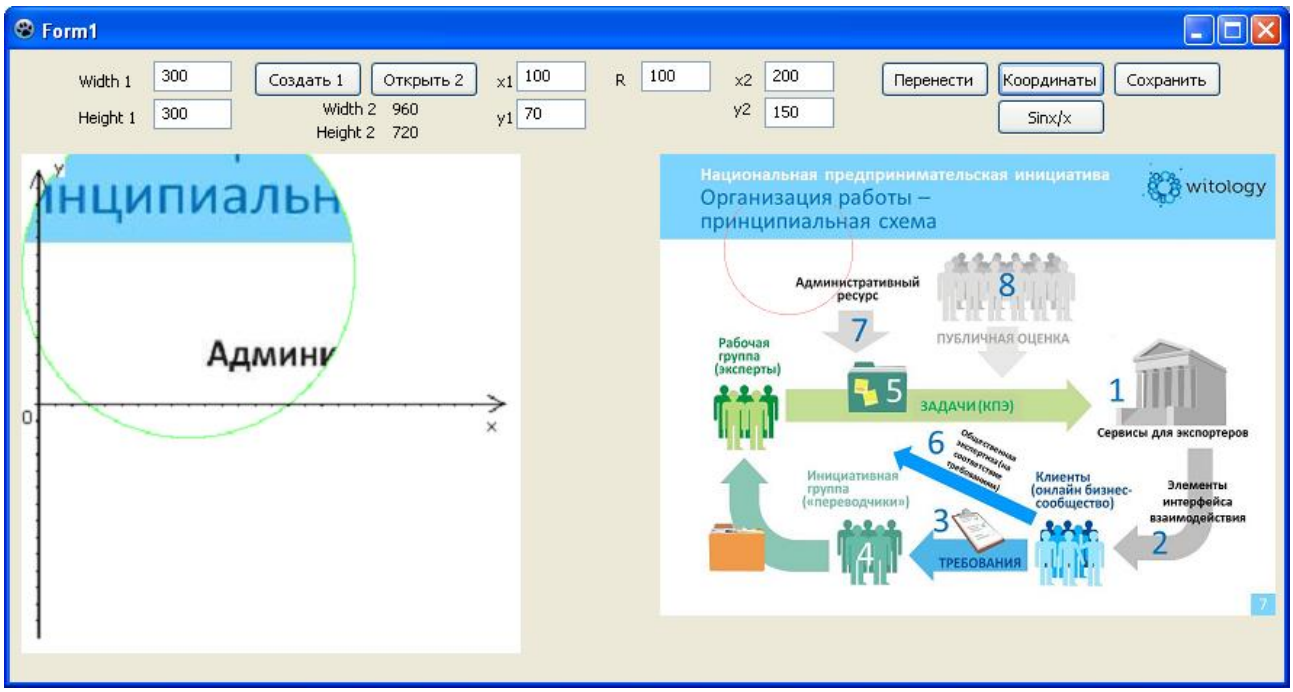


Рис. 15. Вывод координатных осей

Рисуем график функции (рис. 16).

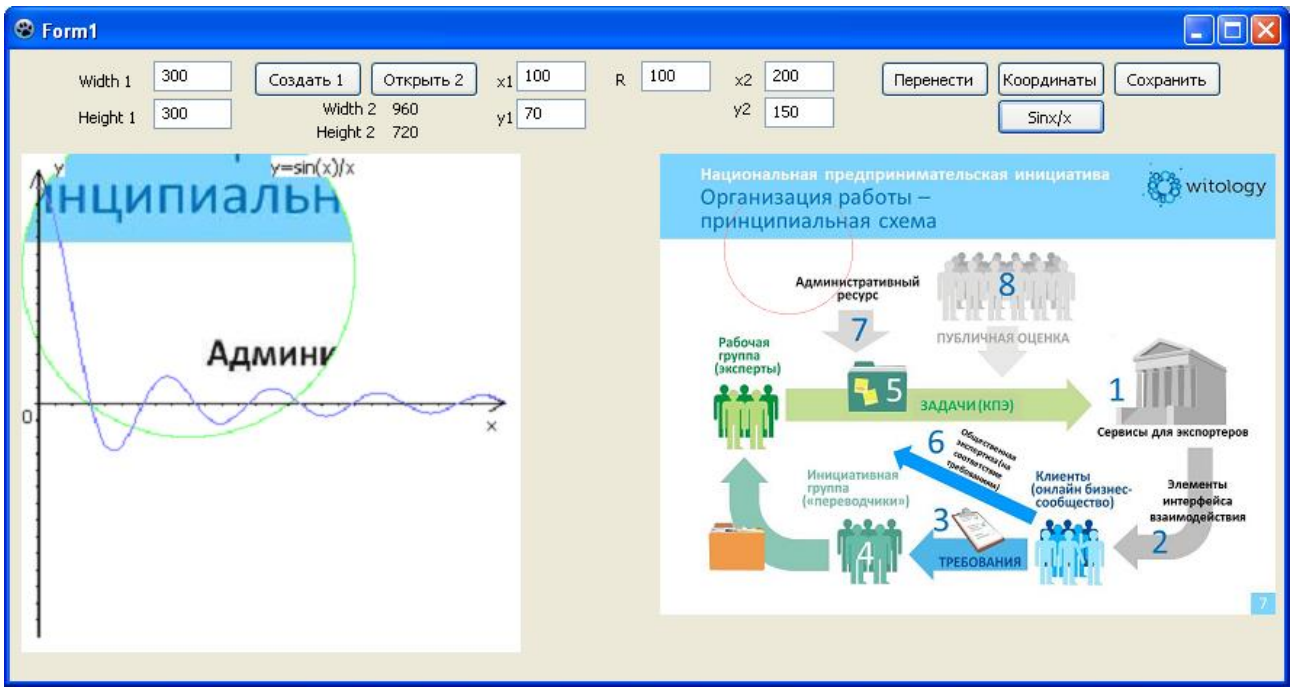


Рис. 16. Вывод графика функции

Сохраняем результат в файл (рис. 17).

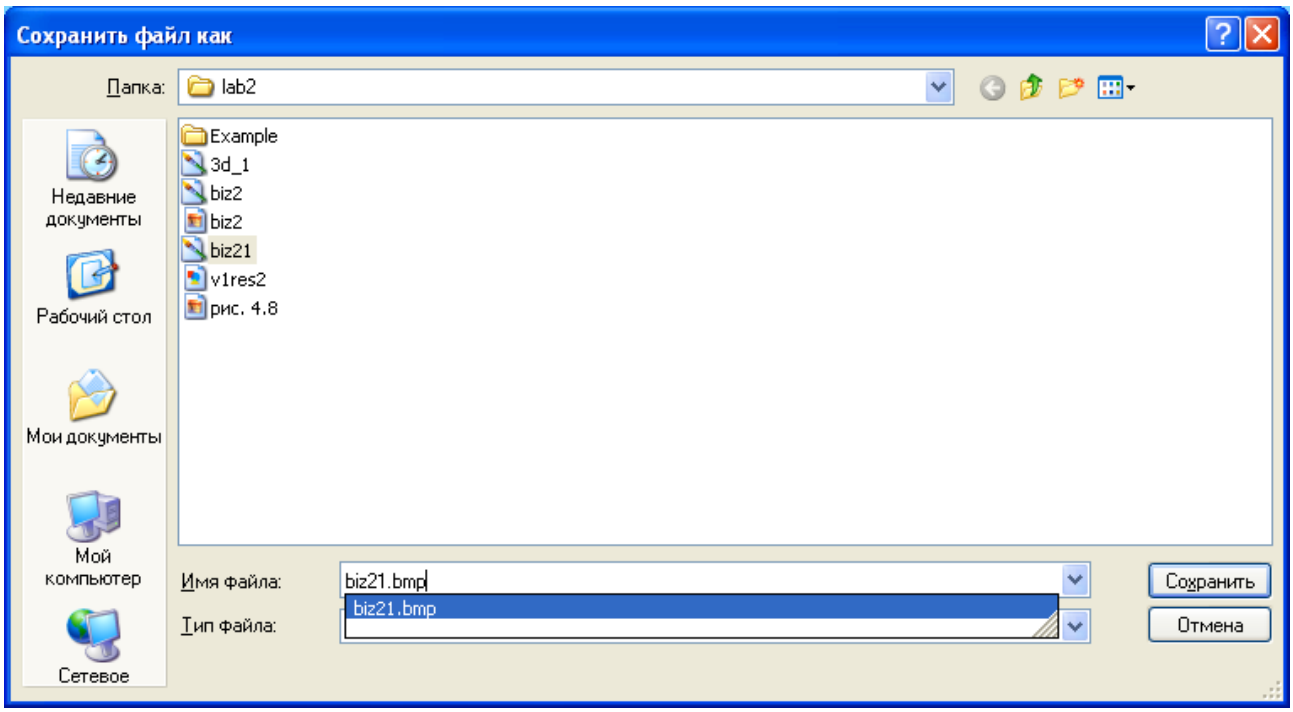


Рис. 17. Диалоговое окно сохранения файла изображения

Смотрим на результат (рис. 18).

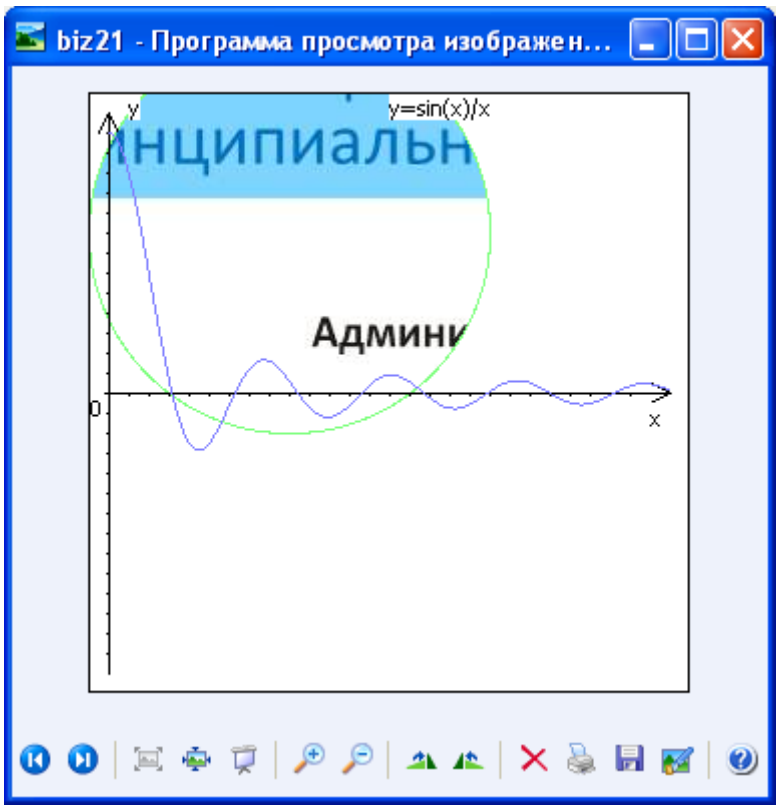


Рис. 18. Сохранённый файл изображения во встроенной программе просмотра изображений